4. GCP and BigQuery – Interview Notes

Quick brush-up notes for tomorrow's interview.

Section	Topic	Priority
A	BigQuery fundamentals	High
B	BigQuery optimization	Very high
C	BigQuery development (SQL scripting, MERGE)	High
D	GCP services (GCS, IAM, Composer, Dataflow, Pub/Sub...)	Medium
E	Important interview questions	Very high

How to use these notes: read the "Remember" lines and the interview answers first, then go through the examples. If you only have 2 hours, read B, then E, then C (MERGE and scripting), then A.

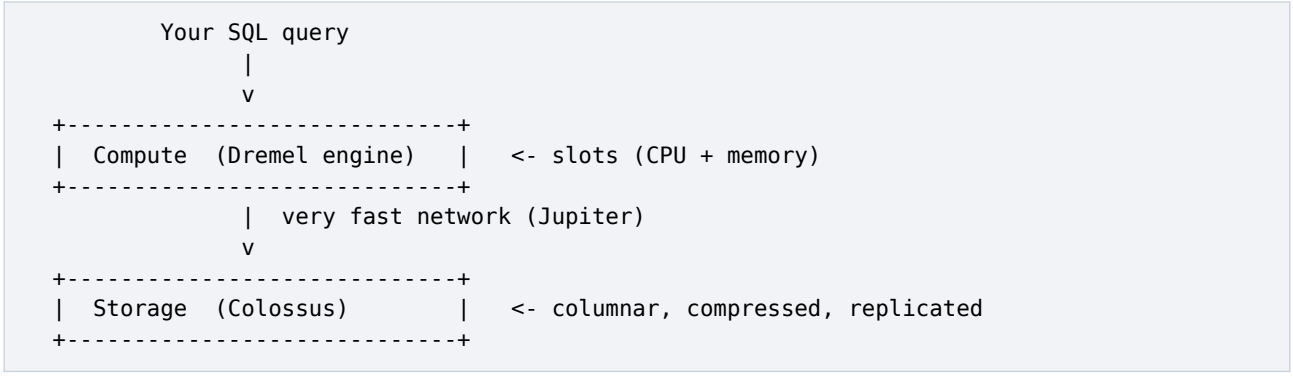
Running example used throughout: a bank loads customer transactions from Teradata into BigQuery, via GCS, using IICS and Composer.

A. BigQuery Fundamentals

1. BigQuery Architecture

BigQuery is Google's serverless, fully managed, columnar data warehouse. You do not manage servers, indexes or storage. You write SQL and Google runs it.

The most important idea: storage and compute are separated.



Key components (just know the names and one-line roles):

- Dremel: the distributed query engine. Splits a query into stages and runs them in parallel.
- Colossus: Google's distributed file system where table data is stored (columnar format, compressed, encrypted).
- Jupiter: the high-speed network connecting compute and storage.
- Borg: the cluster manager that allocates compute.

Why does this matter?

- You can scale storage and compute independently.
- You pay for storage and for queries separately.

- Many users can query the same data at the same time without affecting storage.

Why is it fast for analytics? BigQuery stores data by column. A query reading 2 columns out of 100 only reads those 2 columns.

Remember: BigQuery is built for OLAP (analytics), not OLTP (frequent single-row transactions).

Interview answer:

"BigQuery is a serverless columnar data warehouse that separates storage from compute. Data is stored in Colossus in a compressed columnar format, and queries are executed by the Dremel engine using slots, which are units of compute. Because they are decoupled, storage and compute scale independently and we don't manage any infrastructure."

2. Datasets and Tables

BigQuery resource hierarchy:

```
Project  (billing + permissions boundary)
  └── Dataset  (a container for tables; has a location and access controls)
        ├── Table
        ├── View
        └── Routine (stored procedure / function)
```

Fully qualified table name:

```
`my-project.banking_dwh.fact_transaction`
```

Points to remember:

- A dataset has a fixed location (for example asia-south1 or US). You cannot query across datasets in different locations in one query.
- Access can be controlled at project, dataset, table, and even column or row level.
- Table types: native (BigQuery managed) tables, external tables, views, materialized views, snapshots and clones.

Example:

```
CREATE TABLE banking_dwh.fact_transaction (
  transaction_id  INT64,
  customer_id     STRING,
  amount          NUMERIC,
  txn_date        DATE
);
```

Useful data types: INT64, NUMERIC / BIGNUMERIC (use for money, avoids float errors), FLOAT64, STRING, BOOL, DATE, DATETIME, TIMESTAMP, ARRAY, STRUCT, JSON.

Interview tip: Mention that BigQuery supports nested and repeated fields (STRUCT and ARRAY). Denormalizing with nested data can avoid expensive joins.

3. Views and Materialized Views

View:

A view is a saved SQL query. It stores no data. Every time you query the view, the underlying query runs again.

```
CREATE VIEW banking_dwh.v_high_value_txn AS
SELECT transaction_id, customer_id, amount
FROM banking_dwh.fact_transaction
WHERE amount > 100000;
```

Materialized view:

A materialized view stores the precomputed result of a query. BigQuery refreshes it automatically (incrementally) when the base table changes.

```
CREATE MATERIALIZED VIEW banking_dwh.mv_daily_total AS
SELECT txn_date, SUM(amount) AS total_amount
FROM banking_dwh.fact_transaction
GROUP BY txn_date;
```

View	Materialized View
Stores only the query	Stores the computed result
Runs the full query every time	Reads precomputed data (and only the new changes)
Always current	Automatically refreshed, may be slightly behind
No storage cost	Has storage cost
Supports any SQL	Supports a limited set of SQL
Good for abstraction and security	Good for repeated aggregations

Extra points:

- BigQuery can automatically rewrite queries against the base table to use a matching materialized view (smart tuning).
- Authorized views let you share part of a table with users who have no access to the underlying table.
- Materialized views have restrictions, for example non-deterministic functions are not allowed.

Remember: View = saved query, no speed gain. Materialized view = saved result, faster and cheaper for repeated aggregations.

4. External Tables

An external table lets BigQuery query data that stays outside BigQuery, mostly in GCS (also Google Drive and Bigtable), without loading it.

```
CREATE EXTERNAL TABLE banking_dwh.ext_txn_files
OPTIONS (
  format = 'PARQUET',
  uris = ['gs://bank-landing/txn/*.parquet']
);
```

Native table	External table
Data stored in BigQuery	Data stays in GCS
Fastest	Slower
Full features (clustering, partitioning, caching)	Limited features
Pay for BigQuery storage	Pay for GCS storage

When to use external tables:

- Quick look at landed files before loading.
- Data that is rarely queried.
- A staging pattern: query the external table, validate, then `INSERT INTO ... SELECT` into a native table.

Interview answer:

"I use external tables for ad hoc access or validation of files in GCS. For regular analytics I load the data into native BigQuery tables because they give better performance and support partitioning and clustering."

5. Temporary Tables

Temporary tables hold intermediate results and disappear automatically.

Two kinds:

- Anonymous result tables: every query result is stored in a hidden table for about 24 hours (this is what powers the query cache).
- Script/session temp tables: created with `CREATE TEMP TABLE`. They live only for the script or session.

```
CREATE TEMP TABLE tmp_latest AS
SELECT *
FROM banking_dwh.stg_customer
QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY update_ts DESC) = 1;
```

Use them in scripts and stored procedures to break a large transformation into simple steps, for example a staging step before MERGE.

6. Managed Storage

BigQuery manages storage for you:

- Data is stored in columnar format, compressed and encrypted by default (at rest and in transit).
- Replicated automatically for durability.
- Storage billing: active storage (modified in the last 90 days) and long-term storage (not modified for 90 days, cheaper automatically).
- You can choose logical or physical storage billing at dataset level. Logical bills uncompressed bytes. Physical bills compressed bytes at a higher rate, so it can be cheaper for well-compressed data.

Recovery features:

Feature	What it does
Time travel	Query or restore data as it was in the past (up to 7 days, configurable)
Fail-safe	Extra 7 days of Google-only recovery after time travel ends
Table snapshot	Read-only point-in-time copy of a table
Table clone	Writable lightweight copy, pays only for changed data

Example of time travel:

```
SELECT *
FROM banking_dwh.dim_customer
FOR SYSTEM_TIME AS OF TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 HOUR);
```

Interview use case: "A bad load changed customer records. I used time travel to query yesterday's data and restore the correct version."

7. Compute and Storage Separation

This is a favourite interview question.

Traditional warehouse (for example Teradata)	BigQuery
Storage and compute tied together on the same nodes	Storage and compute are independent
To get more compute, you often add nodes (and storage)	Scale compute without touching storage
Capacity planning needed	Serverless, no capacity planning
You pay for the cluster even when idle	Pay for storage + queries (or slots)

Benefits:

1. Independent scaling.
2. Multiple teams can query the same data without competing for the same disks.
3. Cost control: store a lot of data cheaply, pay for compute only when querying.

Interview answer:

"In BigQuery, data lives in Colossus and queries run on separate compute resources called slots. This separation lets us scale each independently, avoid capacity planning, and pay only for what we store and process."

8. Slots and Execution Basics

A slot is BigQuery's unit of compute capacity (think of it as a virtual worker with CPU and memory). A query is split into stages and many slots work on those stages in parallel.

```
Query
  |
  v
Stage 1: scan + filter    (many slots in parallel)
  |
  v
Stage 2: shuffle + join   (data redistributed between slots)
  |
  v
Stage 3: aggregate
  |
  v
Final result
```

Two ways to pay for compute:

Model	How it works
On-demand	Pay per TiB of data scanned (about $6.25 per TiB in US regions, first 1 TiB per month free). Uses a shared pool of slots.
Capacity pricing (Editions)	Pay for slots per hour (Standard, Enterprise, Enterprise Plus), with autoscaling and optional commitments.

Note: prices change and vary by region, so say "around" and "check the current pricing page" in an interview. The old flat-rate model has been replaced by Editions.

Important terms:

- Slot time (slot-ms): total compute used by a query.
- Shuffle: moving data between stages, for example for joins and GROUP BY.
- Slot contention: many queries competing for slots, causing slowness.

B. BigQuery Optimization

★ Very important

This is the most asked area. The core idea is simple: BigQuery is columnar and (on-demand) you pay for bytes read. So to make queries cheaper and faster, read less data.

1. Partitioning

Partitioning splits a table into segments (partitions) based on a column. BigQuery can then read only the partitions you need.

Types of partitioning:

- Time-unit column: partition on a DATE, TIMESTAMP or DATETIME column (daily, hourly, monthly or yearly).
- Ingestion time: based on when the row was loaded (pseudo column `_PARTITIONTIME`).
- Integer range: partition on an integer column using start, end and interval.

Example:

```
CREATE TABLE banking_dwh.fact_transaction (
  transaction_id INT64,
  customer_id    STRING,
  amount         NUMERIC,
  txn_date       DATE
)
PARTITION BY txn_date
OPTIONS (
  require_partition_filter = TRUE,
  partition_expiration_days = 1095
);
```

Picture:

```
fact_transaction
  ├── 2026-11-01  (partition)
  ├── 2026-11-02  (partition)
  ├── 2026-11-03  (partition)
  └── 2026-11-04  (partition)
```

Key facts:

- A table can have up to 10,000 partitions.
- `require_partition_filter = TRUE` forces users to filter on the partition column, preventing accidental full scans.
- `partition_expiration_days` automatically deletes old partitions.
- A table can have only one partitioning column.
- Best for large tables with a column commonly used in filters, usually a date.

2. Partition Pruning

Partition pruning means BigQuery skips (does not read) partitions that do not match your filter.

```
SELECT SUM(amount)
FROM banking_dwh.fact_transaction
WHERE txn_date = '2026-11-04';
```

BigQuery reads only the 2026-11-04 partition, not the whole table.

Pruning stops working when:

- You do not filter on the partition column.
- You wrap the partition column in a function or expression that BigQuery cannot evaluate in advance. Prefer filters like `txn_date BETWEEN '2026-11-01' AND '2026-11-04'`.
- The filter value depends on a subquery that is not constant (some cases do not prune well).

Check pruning: use a dry run or look at "bytes processed" before running.

Example:

Query	Bytes processed
No date filter	2 TB (full table)
Filter on one day	5 GB

3. Clustering

Clustering sorts the data inside the table (or inside each partition) by up to four columns. BigQuery then skips blocks of data that cannot contain your filter values.

```
CREATE TABLE banking_dwh.fact_transaction (
  transaction_id INT64,
  customer_id    STRING,
  branch_id      STRING,
  amount         NUMERIC,
  txn_date       DATE
)
PARTITION BY txn_date
CLUSTER BY customer_id, branch_id;
```

Key facts:

- Up to 4 clustering columns. The order matters: filter on the first column gives the most benefit.
- Works with or without partitioning.
- Good for columns with high cardinality (many distinct values), such as customer_id.
- BigQuery re-clusters data automatically in the background at no extra charge.
- The cost estimate before running a clustered query is an upper bound. The actual bytes billed can be lower.

4. Partitioning vs Clustering

★ Very important

Partitioning	Clustering
Divides table into distinct segments	Sorts data inside the table or partition

Partitioning	Clustering
One column only	Up to 4 columns
Best for date or low-cardinality columns	Good for high-cardinality columns
Cost known before the query runs (exact pruning)	Cost estimate is an upper bound
Limit of 10,000 partitions	No such partition limit
Can set partition expiration	No expiration setting

Rule of thumb:

- Partition on the column you almost always filter by with a range or date (for example txn_date).
- Cluster on columns used in filters, joins or GROUP BY (for example customer_id, branch_id).
- Use both together on big tables: partition by date, cluster by customer_id.

Tip: Do not partition small tables (under about 1 GB). Too many tiny partitions add metadata overhead and bring little benefit.

5. Query Execution Plan

After a query runs, the Execution details tab (in the console) shows the query plan: the stages BigQuery used.

What to look at:

- Stage with the highest time or slot time (the bottleneck).
- Records read vs records written for each stage (did a join explode the rows?).
- Shuffle size (large shuffle means heavy data movement).
- Spill to disk (memory not enough, often for big joins or sorts).
- Skew: one worker doing much more work than others (usually a hot join or GROUP BY key).

You can also get job details from `INFORMATION_SCHEMA.JOBS`.

Interview answer:

"I open the execution details to find the stage consuming most time, check bytes read and shuffled, and look for skew or spill. Then I fix that specific bottleneck, for example by pruning partitions, filtering earlier, or changing the join."

6. Bytes Processed

On-demand pricing is based on bytes processed. Always check this before and after a query.

- The console shows "This query will process X GB" before you run it.
- Use a dry run to estimate without running: `bq query --dry_run`.
- Use `maximum_bytes_billed` to fail a query that would scan too much.

Important facts:

- Only the columns you select are read, so fewer columns means fewer bytes.
- `LIMIT` does NOT reduce bytes processed on a normal non-clustered table. BigQuery still scans the data. Use partition filters instead.
- Cached results are free (if the same query runs again on unchanged data, within about 24 hours).
- There is a minimum billing of 10 MB per table referenced.

- Previewing a table in the console is free, so use Preview instead of `SELECT * ... LIMIT 10`.

7. Query Cost Optimization

A checklist you can say in the interview:

1. Select only the columns you need.
2. Filter on the partition column (partition pruning).
3. Cluster on commonly filtered columns.
4. Use materialized views for repeated aggregations.
5. Filter early, join later (reduce rows before the join).
6. Avoid joining huge tables when not needed; join on partition/cluster keys.
7. Use approximate functions where exactness is not needed, for example `APPROX_COUNT_DISTINCT`.
8. Use `require_partition_filter` and `maximum_bytes_billed` as guardrails.
9. Set table or partition expiration for temporary or old data.
10. For heavy sustained workloads, consider capacity pricing (Editions) rather than on-demand.
11. Avoid repeated single-row DML. Prefer batch loads and MERGE.
12. Use nested and repeated fields (denormalized) instead of expensive joins when it fits the model.

8. Materialized View Optimization

Materialized views reduce cost and latency when many users run the same aggregation.

Example: management dashboards repeatedly compute daily totals from a 5 TB table.

```
Without MV:  every dashboard load scans the large base table
With MV:     dashboard reads a small precomputed result and only new changes are processed
```

Points to remember:

- Best for aggregations (SUM, COUNT, AVG) and filters on large, frequently queried tables.
- BigQuery refreshes them automatically and incrementally. You can also set a refresh interval.
- Queries on the base table can be automatically rewritten to use the materialized view.
- Not suitable when the SQL is too complex for MV restrictions or data changes extremely often.

9. Avoiding SELECT *

Because BigQuery is columnar, `SELECT *` reads every column.

```
-- Bad: reads all columns
SELECT * FROM banking_dwh.fact_transaction;

-- Good: reads only 2 columns
SELECT transaction_id, amount FROM banking_dwh.fact_transaction;
```

If a table has 100 columns and you need 3, `SELECT *` can scan around 30 times more data than needed.

Handy option when you need almost all columns:

```
SELECT * EXCEPT (large_json_column) FROM banking_dwh.fact_transaction;
```

Remember: `LIMIT` does not make `SELECT *` cheaper.

C. BigQuery Development

1. BigQuery Scripting

Scripting lets you run multiple SQL statements together with variables, control flow and error handling.

```
DECLARE run_date DATE DEFAULT CURRENT_DATE();
DECLARE row_count INT64;

SET row_count = (
  SELECT COUNT(*) FROM banking_dwh.stg_txn WHERE txn_date = run_date
);

IF row_count = 0 THEN
  SELECT 'No data today' AS message;
ELSE
  INSERT INTO banking_dwh.fact_transaction
  SELECT * FROM banking_dwh.stg_txn WHERE txn_date = run_date;
END IF;
```

Main building blocks:

- `DECLARE` and `SET` for variables.
- `IF ... ELSE`, `WHILE`, `LOOP`, `FOR` for control flow.
- `BEGIN ... EXCEPTION WHEN ERROR THEN ... END` for error handling.
- `CREATE TEMP TABLE` for intermediate steps.
- `RAISE` to raise an error.

2. Stored Procedures

A stored procedure is a saved script that you can call with parameters.

```
CREATE OR REPLACE PROCEDURE banking_dwh.sp_load_txn(IN load_date DATE)
BEGIN
  DELETE FROM banking_dwh.fact_transaction WHERE txn_date = load_date;

  INSERT INTO banking_dwh.fact_transaction
  SELECT * FROM banking_dwh.stg_txn WHERE txn_date = load_date;
END;

CALL banking_dwh.sp_load_txn(DATE '2026-11-04');
```

Parameter modes: `IN` (input), `OUT` (output), `INOUT` (both).

Why use them?

- Reusable ETL logic.
- Easy to call from Composer/Airflow or schedulers.
- Central place for load logic and error handling.

Interview tip: Show how the procedure above is rerunnable (delete the day's data, then insert). That is the idempotency idea from ETL.

3. Dynamic SQL and 4. EXECUTE IMMEDIATE

Dynamic SQL means building a SQL statement as a string and running it at runtime. In BigQuery you use `EXECUTE IMMEDIATE`.

Use it when table names, columns or filters change at runtime, for example loading many tables with one script.

```
DECLARE table_name STRING DEFAULT 'fact_transaction';
DECLARE load_date  DATE   DEFAULT DATE '2026-11-04';

EXECUTE IMMEDIATE FORMAT("""
  SELECT COUNT(*) AS cnt
  FROM banking_dwh.%s
  WHERE txn_date = @d
""", table_name)
USING load_date AS d;
```

Points to remember:

- Values can be passed safely with parameters (`USING ... AS name`). This prevents SQL injection.
- Table and column names cannot be parameters. Build them into the string with `FORMAT`.
- Capture results with `INTO`: `EXECUTE IMMEDIATE "SELECT COUNT(*) FROM t" INTO row_count;`

Use dynamic SQL carefully: it is harder to read and debug.

5. Temporary Tables

Covered in Section A. In development, the pattern is:

1. Load raw data into a staging table (or temp table).
2. Clean and deduplicate in a temp table.
3. MERGE into the final table.

6. MERGE

★ Frequently asked

MERGE does insert, update and delete in one statement. It is the standard way to implement incremental loads and SCD logic in BigQuery.

Example: upsert customers.

```
MERGE banking_dwh.dim_customer AS T
USING banking_dwh.stg_customer AS S
ON T.customer_id = S.customer_id
WHEN MATCHED AND (T.city != S.city OR T.name != S.name) THEN
  UPDATE SET
    name = S.name,
    city = S.city,
    updated_at = CURRENT_TIMESTAMP()
WHEN NOT MATCHED THEN
  INSERT (customer_id, name, city, updated_at)
  VALUES (S.customer_id, S.name, S.city, CURRENT_TIMESTAMP());
```

Clauses:

Clause	Meaning
WHEN MATCHED	Key exists in target: update or delete
WHEN NOT MATCHED (BY TARGET)	In source but not in target: insert
WHEN NOT MATCHED BY SOURCE	In target but not in source: delete or mark inactive

The most common MERGE error:

"UPDATE/MERGE must match at most one source row for each target row."

This happens when the source has duplicates for the same key. Fix: deduplicate the staging data first.

```
USING (
  SELECT *
  FROM banking_dwh.stg_customer
  QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY update_ts DESC) = 1
) AS S
```

Performance tip: if the target is partitioned, add a filter on the partition column in the ON condition or in the source to limit the partitions scanned.

Because MERGE is idempotent (rerunning the same batch does not create duplicates), it also supports safe restarts.

7. INFORMATION_SCHEMA

INFORMATION_SCHEMA contains read-only views of metadata. You query it with SQL.

Useful ones:

View	Use
dataset.INFORMATION_SCHEMA.TABLES	List tables and types
dataset.INFORMATION_SCHEMA.COLUMNS	Column names and data types
dataset.INFORMATION_SCHEMA.PARTITIONS	Partition row counts and last modified times
region-xx.INFORMATION_SCHEMA.JOBS_BY_PROJECT	Query history, bytes billed, slot time, errors
region-xx.INFORMATION_SCHEMA.TABLE_STORAGE	Storage size per table

Example: find the most expensive queries from the last 7 days.

```
SELECT
  user_email,
  job_id,
  total_bytes_billed / POW(1024, 4) AS tib_billed,
  total_slot_ms / 1000 AS slot_seconds,
  query
FROM `region-us`.INFORMATION_SCHEMA.JOBS_BY_PROJECT
WHERE creation_time > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 7 DAY)
  AND job_type = 'QUERY'
ORDER BY total_bytes_billed DESC
LIMIT 10;
```

Example: check partition row counts for reconciliation.

```
SELECT partition_id, total_rows
FROM banking_dwh.INFORMATION_SCHEMA.PARTITIONS
WHERE table_name = 'fact_transaction'
ORDER BY partition_id DESC;
```

8. Exception Handling

Use `BEGIN ... EXCEPTION WHEN ERROR THEN ... END` inside scripts and procedures.

```
BEGIN
  MERGE banking_dwh.dim_customer T
  USING banking_dwh.stg_customer S
  ON T.customer_id = S.customer_id
  WHEN MATCHED THEN UPDATE SET city = S.city
  WHEN NOT MATCHED THEN INSERT (customer_id, city) VALUES (S.customer_id, S.city);

EXCEPTION WHEN ERROR THEN
  INSERT INTO banking_dwh.etl_error_log (job_name, error_message, logged_at)
  VALUES ('load_customer', @@error.message, CURRENT_TIMESTAMP());

  RAISE USING MESSAGE = @@error.message;   -- fail the job so the scheduler knows
END;
```

Points:

- `@@error.message` holds the error text; `@@error.statement_text` the failing statement.
- Log the error in a control table, then re-raise it so Composer or the scheduler marks the task as failed. Do not silently swallow errors.
- Combine with batch status tables (RUNNING, SUCCESS, FAILED) for restartability.

9. Transaction Basics

BigQuery supports multi-statement transactions inside a script or session.

```
BEGIN TRANSACTION;

DELETE FROM banking_dwh.fact_transaction WHERE txn_date = '2026-11-04';
INSERT INTO banking_dwh.fact_transaction
SELECT * FROM banking_dwh.stg_txn WHERE txn_date = '2026-11-04';

COMMIT TRANSACTION;
```

If something fails, use `ROLLBACK TRANSACTION`.

Points to remember:

- Each DML statement is atomic. Multi-statement transactions let several statements succeed or fail together.
- BigQuery uses snapshot isolation.
- Transactions can span multiple tables.
- BigQuery is not an OLTP database. Avoid thousands of tiny single-row DML statements. Prefer batch loads or MERGE.
- DML has concurrency and quota limits. Heavy concurrent updates to the same table may queue or conflict.

D. GCP Services

1. Google Cloud Storage (GCS)

GCS is Google's object storage. In data engineering it is the landing zone for files before they are loaded to BigQuery.

Typical flow:

```
Teradata / Oracle  -->  IICS or export job  -->  GCS (landing)  -->  BigQuery (staging)  -->  BigQuery (f
```

Why use GCS as a landing area?

- Cheap and durable storage.
- Keeps the raw files for audit, replay and reload.
- Decouples extraction from loading.

2. GCS Bucket and Object Structure

```
Bucket:  bank-landing-prod      (globally unique name, has a location)
  └── Objects (files)
        txn/2026/11/04/txn_part_001.parquet
        txn/2026/11/04/txn_part_002.parquet
        customer/2026/11/04/customer.csv
```

Key points:

- A bucket is the top-level container. Bucket names are globally unique.
- An object is a file plus its metadata.
- The namespace is flat. "Folders" are just prefixes in the object name (`txn/2026/11/04/`).
- Objects are immutable: to change a file you overwrite it with a new version.
- Common path convention: `source/table/yyyy/mm/dd/file`.
- Lifecycle rules can move or delete files automatically (for example, delete raw files after 90 days).
- Object versioning can keep old versions.

3. Storage Classes

Class	Typical use	Minimum storage duration
Standard	Frequently accessed, active data and landing files	None
Nearline	Accessed about once a month	30 days
Coldline	Accessed about once a quarter	90 days
Archive	Long-term retention, compliance	365 days

All classes have the same low latency access and high durability. Colder classes cost less to store but more to retrieve, and charge early deletion fees.

Tip: Landing files = Standard first, then a lifecycle rule moves old files to Nearline or Coldline. Autoclass can do this automatically.

4. IAM and Service Accounts

IAM (Identity and Access Management) answers: who can do what on which resource.

- Principal: user, group or service account.
- Role: a collection of permissions.
- Resource: project, dataset, bucket and so on.

A service account is an identity for an application or pipeline (not a person). IICS Secure Agent, Composer and Dataflow typically run using service accounts.

Common roles:

Role	What it gives
roles/bigquery.jobUser	Permission to run queries and load jobs in a project
roles/bigquery.dataViewer	Read table data
roles/bigquery.dataEditor	Read and write table data
roles/bigquery.admin	Full BigQuery administration
roles/storage.objectViewer	Read objects in GCS
roles/storage.objectAdmin	Read, write, delete objects in GCS

Remember: to run a query you usually need `jobUser` (to run the job) plus `dataViewer` on the data.

Best practices:

- Least privilege: give only the roles needed.
- Use separate service accounts per pipeline or environment (dev, pre-prod, prod).
- Grant roles at the lowest sensible level (dataset or bucket rather than the whole project).
- Avoid downloading and sharing service account keys. Prefer attached service accounts or workload identity.

5. GCS to BigQuery Loading

★ Frequently asked

Ways to load files from GCS to BigQuery:

1. `bq load` command line.
2. `LOAD DATA` SQL statement.
3. Console or API (load job).
4. External table, then `INSERT INTO ... SELECT`.
5. BigQuery Data Transfer Service (scheduled transfers).
6. Orchestrated by Composer (`GCSToBigQueryOperator`) or by an ETL tool such as IICS.

Example with SQL:

```
LOAD DATA INTO banking_dwh.stg_txn
FROM FILES (
  format = 'PARQUET',
  uris = ['gs://bank-landing-prod/txn/2026/11/04/*.parquet']
);
```

Example with bq:

```
bq load --source_format=PARQUET \
  banking_dwh.stg_txn \
  "gs://bank-landing-prod/txn/2026/11/04/*.parquet"
```

Important facts:

- Batch load jobs from GCS are free (no query charge). You pay for storage.
- Wildcards (`*`) load many files in one job.
- Write dispositions: `WRITE_APPEND` (add), `WRITE_TRUNCATE` (replace), `WRITE_EMPTY` (fail if table has data).
- Loads are atomic: the job either loads all data or fails.
- Schema: provide explicitly in production. Autodetect is convenient but can guess types wrongly.
- GCS bucket and BigQuery dataset must be in compatible locations.

Recommended pattern:

```
GCS file  -->  staging table (raw)  -->  validate + dedupe  -->  MERGE into final table  -->  reconcile c
```

6. CSV, JSON, Avro and Parquet

Format	Type	Schema included	Notes
CSV	Row, text	No	Simple, widely used. Watch delimiters, quotes, headers, nulls, date formats. Slowest and largest.
JSON	Row, text	No	Must be newline-delimited JSON for BigQuery. Good for nested data.
Avro	Row, binary	Yes	Compact, schema embedded, fast to load. Good for streaming and schema evolution.
Parquet	Columnar, binary	Yes	Compressed, efficient for analytics. Excellent for large batch loads.

Which to choose?

- Parquet or Avro for production pipelines: smaller, faster, keeps data types.
- CSV when the source system can only export CSV. Be careful with data types and special characters.

Interview tip: "I prefer Parquet or Avro because the schema travels with the data, which reduces type errors, and they are compressed and load efficiently."

7. Cloud Composer / Airflow

Cloud Composer is Google's managed Apache Airflow service. It is used to orchestrate and schedule pipelines.

Key Airflow concepts:

- DAG: a workflow defined in Python, with tasks and dependencies.
- Task: one unit of work.
- Operator: a template for a task (for example `BigQueryInsertJobOperator`, `GCSToBigQueryOperator`, `BashOperator`, `PythonOperator`).

- Schedule: cron-like timing (for example daily at 2 AM).
- Retries, alerts, SLAs and backfills (re-running past dates).

Example flow:

```
check_file_in_gcs  -->  load_to_staging  -->  validate_counts  -->  merge_to_target  -->  send_success_em
```

Simple DAG idea:

```
with DAG("daily_txn_load", schedule="0 2 * * *", start_date=datetime(2026, 11, 1), catchup=False) as dag:
    load = GCSToBigQueryOperator(task_id="load_stg", ...)
    merge = BigQueryInsertJobOperator(task_id="merge_target", ...)
    load >> merge
```

Composer vs IICS/Control-M: Composer (Airflow) is a cloud-native orchestrator in code. IICS Taskflow is Informatica's own orchestration. Control-M is an enterprise scheduler. They can work together: for example Control-M or Composer triggers an IICS job.

8. Dataflow

Dataflow is a fully managed service for running Apache Beam pipelines for both batch and streaming data.

Use it when you need:

- Real-time stream processing (for example Pub/Sub to Dataflow to BigQuery).
- Complex transformations at large scale.
- The same code for batch and streaming.

It autoscales workers and you pay for the resources used.

9. Dataproc

Dataproc is a managed service for Apache Spark and Hadoop clusters.

Use it when:

- You have existing Spark/Hadoop jobs to migrate to GCP.
- You need Spark libraries or custom big-data processing.

Clusters can be created quickly and deleted after the job (ephemeral clusters) to save cost. There is also a serverless Spark option.

Simple way to remember:

Service	Remember as
Dataflow	Managed Beam: batch + streaming pipelines
Dataproc	Managed Spark/Hadoop
Composer	Managed Airflow: orchestration
Pub/Sub	Messaging / event ingestion
BigQuery	Data warehouse
GCS	Object storage / landing zone

10. Pub/Sub

Pub/Sub is a fully managed messaging service for real-time event ingestion. It decouples senders (publishers) from receivers (subscribers).

```
Publisher (app, CDC tool)  -->  Topic  -->  Subscription  -->  Subscriber (Dataflow / BigQuery)
```

Concepts:

- Topic: where messages are published.
- Subscription: a consumer's view of a topic. Each subscription receives a copy of the messages.
- Acknowledgement (ack): the subscriber confirms a message was processed; unacked messages are redelivered.
- Delivery is at-least-once by default, so consumers must handle possible duplicates (idempotent processing).

Example: card transactions arrive as events in Pub/Sub, a Dataflow job processes them, and the results land in BigQuery for near real-time analytics. Pub/Sub also has a direct BigQuery subscription option.

11. Cloud Logging

Cloud Logging collects and stores logs from GCP services (BigQuery, Composer, Dataflow, GCS and so on).

Useful for ETL support:

- Search logs to troubleshoot a failed job (filter by resource, severity, job ID).
- Create log-based metrics and alerts (with Cloud Monitoring), for example alert when a Composer task fails.
- Route logs to BigQuery or GCS using sinks for long-term analysis and audit.
- Audit logs show who accessed or changed data.

Interview answer:

"When a pipeline fails, I check the Composer task logs or job details first, then Cloud Logging for detailed error messages, and BigQuery INFORMATION_SCHEMA.JOBS for query-level errors. I set up alerts so failures notify the support team."

E. Important Interview Questions

Q1. What happens when you execute a BigQuery query?

Answer:

1. Parse and validate: BigQuery checks syntax, tables, columns and your permissions.
2. Cache check: if the same query ran recently on unchanged data, the cached result is returned for free.
3. Planning: the optimizer creates a plan and decides which partitions and columns to read (partition pruning, column pruning).
4. Execution: Dremel splits the work into stages. Slots read data from Colossus in parallel, then shuffle data between stages for joins, GROUP BY and sorts.

5 Result: final results are written to a temporary (anonymous) table and returned to you; this table backs the cache.

6 Billing: you are billed for bytes processed (on-demand) or slot usage (editions).

```
SQL -> parse/validate -> cache? -> plan -> slots read columns/partitions -> shuffle -> aggregate -> resul
```

Interview answer:

"BigQuery validates the query, checks the cache, then builds an execution plan. The Dremel engine splits it into stages, and slots process data in parallel from columnar storage, shuffling data between stages for joins and aggregations. The result is written to a temporary table and cost is calculated based on bytes processed or slot usage."

Q2. How does partitioning reduce query cost?

Answer:

With on-demand pricing, cost depends on bytes read. A partitioned table is divided into segments, and when the query filters on the partition column, BigQuery reads only the matching partitions.

Example: a 2 TB transaction table partitioned by `txn_date`. A query for one day reads only that day's partition (for example 5 GB), not the full 2 TB.

Interview answer:

"Partitioning splits the table by a column such as date. When my query filters on that column, BigQuery prunes the partitions that don't match and scans only the required ones, so the bytes processed and the cost drop significantly. I also enable require_partition_filter to prevent accidental full scans."

Q3. How does clustering differ from partitioning?

Answer:

Partitioning splits the table into separate segments (one column, up to 10,000 partitions) and gives exact pruning and cost estimates. Clustering sorts data inside the table or partition on up to four columns, so BigQuery can skip blocks, which works well for high-cardinality columns. The cost estimate for clustered tables is an upper bound.

Interview answer:

"I partition by date for coarse pruning, and then cluster on columns like customer_id or branch_id that are commonly used in filters and joins. Partitioning removes entire segments, while clustering reduces the data scanned inside those segments. They complement each other."

Q4. How do you load data from GCS into BigQuery?

Answer:

- Land files in GCS (CSV, Avro, Parquet).
- Load into a staging table using `LOAD DATA`, `bq load`, a load job, or GCSToBigQueryOperator in Composer (or IICS).
- Validate: counts, nulls, duplicates, data types.
- Deduplicate and MERGE into the final table.
- Reconcile source vs target counts and log the batch status.
- Archive or move the processed file.

Interview answer:

"I land the files in GCS, then run a load job into a staging table, preferably with Parquet or Avro to keep the schema. After validation and deduplication I MERGE into the final partitioned table, and then reconcile counts. Batch load jobs are free of query charges, and the whole flow is orchestrated by Composer or IICS with logging and retry."

Q5. What is the difference between a view and a materialized view?

Answer:

A view stores only the SQL and runs it every time. A materialized view stores the precomputed result and refreshes it automatically, which makes repeated aggregations faster and cheaper but with limited SQL support and storage cost.

Interview answer:

"A view is just a saved query and gives no performance benefit. A materialized view physically stores the result and is incrementally refreshed, so dashboards and repeated aggregations read much less data. I use views for abstraction and security, and materialized views for frequently repeated aggregations on large tables."

Q6. How would you troubleshoot a slow BigQuery query?

Answer:

1. Check bytes processed. Is it scanning too much? Is `SELECT *` being used? Is partition pruning working?
2. Open the execution details. Find the slowest stage.
3. Look at the join and shuffle: are rows exploding? Is one key skewed? Is data spilling to disk?
4. Check slot availability: is the job waiting for slots (contention)? Compare with `INFORMATION_SCHEMA.JOBS` (slot time, queue time).
5. Check the data layout: is the table partitioned and clustered on the filter and join columns?
6. Optimize: select needed columns, filter early, pre-aggregate before joining, avoid cross joins, use a materialized view or pre-computed table, and avoid repeated subqueries by using temp tables.
7. Compare before and after runtime and bytes processed.

Interview answer:

"I begin with bytes processed and the execution plan to find the bottleneck stage. I check partition pruning, join sizes, shuffle volume, skew and slot contention. Then I apply the specific fix, such as filtering earlier, pruning partitions, clustering, pre-aggregating before a join, or using a materialized view, and I compare runtime and bytes before and after."

Q7. How do you implement incremental loading in BigQuery?

Answer:

Steps:

1. Read the last watermark (for example the last `update_timestamp`) from a control table.
2. Extract only new or changed records from the source (using the watermark or CDC).
3. Land them in GCS or a staging table.
4. Validate and deduplicate the staging data (one row per key).
5. `MERGE` into the target: update existing keys, insert new keys.

6 Reconcile counts and amounts.

7 Update the watermark only after the load succeeds.

8 Log the batch status for restartability.

```
MERGE banking_dwh.dim_customer T
USING (
  SELECT * FROM banking_dwh.stg_customer
  QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY update_ts DESC) = 1
) S
ON T.customer_id = S.customer_id
WHEN MATCHED THEN UPDATE SET name = S.name, city = S.city, update_ts = S.update_ts
WHEN NOT MATCHED THEN INSERT (customer_id, name, city, update_ts)
VALUES (S.customer_id, S.name, S.city, S.update_ts);
```

For large fact tables, another common pattern is partition overwrite: reload only the affected date partitions (delete that partition's data and insert the new data, in one transaction or using `WRITE_TRUNCATE` on a partition).

Interview answer:

"I use a watermark stored in a control table to extract only changed records, land them in staging, deduplicate on the business key, and MERGE into the target. The watermark is advanced only after a successful load, and I reconcile counts afterwards. For large partitioned fact tables I may use partition-level overwrite. Because MERGE is idempotent, reruns do not create duplicates."

F. Last-Minute Revision Sheet

Must-remember facts

Topic	Remember
Architecture	Serverless, columnar, storage and compute separated (Colossus + Dremel)
Slot	Unit of compute capacity
On-demand pricing	Pay per TiB scanned (about $6.25 per TiB, first 1 TiB per month free); check current pricing
Editions	Pay for slots per hour with autoscaling
Partitioning	One column, up to 10,000 partitions, exact pruning
Clustering	Up to 4 columns, sorts data, cost estimate is an upper bound
LIMIT	Does not reduce bytes scanned on normal tables
SELECT *	Reads all columns, avoid it
View	Saved query, no storage
Materialized view	Stored result, automatic incremental refresh
External table	Data stays in GCS, slower, fewer features
Time travel	Up to 7 days of history, then 7 days fail-safe
MERGE	Insert + update + delete in one statement, dedupe the source first
EXECUTE IMMEDIATE	Dynamic SQL, parameters for values, FORMAT for names
Exceptions	BEGIN ... EXCEPTION WHEN ERROR THEN ... END, `@@error.message`
Load from GCS	Free batch load jobs; prefer Parquet or Avro
GCS classes	Standard, Nearline (30d), Coldline (90d), Archive (365d)
Composer	Managed Airflow, DAG of tasks
Dataflow	Managed Apache Beam (batch + streaming)
Dataproc	Managed Spark/Hadoop
Pub/Sub	Messaging, at-least-once delivery
Cloud Logging	Logs, alerts, audit

Typical optimization answer in 30 seconds

"I first check bytes processed and the execution plan. I make sure the query selects only required columns, filters on the partition column for pruning, and uses clustered columns in filters and joins. I filter and aggregate before joins, avoid SELECT *, and use materialized views or pre-aggregated tables for repeated workloads. I compare before and after bytes and runtime."

Typical ETL pipeline answer in 30 seconds

"Data is extracted from Teradata, landed as Parquet in GCS, loaded into a BigQuery staging table, validated and deduplicated, then merged into the final partitioned and clustered table. The pipeline is orchestrated by Composer or IICS with a watermark and batch status in a control table, error logging, and count reconciliation, so reruns are safe and idempotent."

Quick self-test (answer out loud)

1. Why is BigQuery fast for analytics?

2. Does LIMIT reduce cost? Why or why not?
3. When would you choose clustering over partitioning?
4. Why can a MERGE fail with "must match at most one source row"?
5. What is the difference between a view and a materialized view?
6. Why are Parquet and Avro preferred over CSV?
7. How do you make a rerun of a failed load safe?
8. A query is slow. What are your first three checks?

Good luck with your interview tomorrow!